

Big O de memoria + expresiones y operadores

Clase 8 · Martes 6 de octubre de 2026 · Laboratorio

Noticia tech

✦ AI Overview

Ryan Roslansky is stepping down from his executive role at Microsoft and will leave the company at the end of 2026.  CNBC



Key Details

- **Reason for Leaving:** Roslansky chose to depart rather than relocate full-time to Microsoft's headquarters in **Redmond, Washington**, noting that his life and family are rooted in the **San Francisco Bay Area**.  qz.com +1
- **Transition Role:** He will stay at Microsoft through **December 2026** to serve as an advisor to CEO Satya Nadella.  www.entrepreneur.com +1
- **Leadership Changes:** Dan Shapero remains CEO of LinkedIn, and Charles Lamanna is taking over Office and Microsoft Teams responsibilities.  The Verge +1
- **Background:** Roslansky spent nearly 18 years across LinkedIn and Microsoft, leading LinkedIn for six years and expanding its membership from 700 million to 1.3 billion.  LinkedIn · Inna Kuts +2

Vistazo al mundo laboral

Software engineer salaries in Mexico by seniority (2026)

Mexico's software engineering market is mature. Cities like Guadalajara, Monterrey, and Mexico City have deep talent pools built around decades of nearshore outsourcing.

The table below reflects 2026 market rates for full-time employees hired through a compliant payroll structure.

Seniority Level	Monthly Salary (MXN)	Monthly Salary (USD)	Annual Salary (USD)
Junior (0–2 years)	MXN 25,000–40,000	USD 1,389–2,222	USD 16,667–26,667
Mid-Level (2–5 years)	MXN 45,000–70,000	USD 2,500–3,889	USD 30,000–46,667
Senior (5–8 years)	MXN 70,000–100,000	USD 3,889–5,556	USD 46,667–66,667
Staff / Lead (8+ years)	MXN 95,000–130,000	USD 5,278–7,222	USD 63,333–86,667
Engineering Manager	MXN 110,000–160,000	USD 6,111–8,889	USD 73,333–106,667

How does this compare to the U.S.? A mid-level software engineer in San Francisco earns USD 150,000–180,000 per year. The same profile in Mexico costs USD 30,000–47,000. That's a 70–80% cost reduction with no compromise on skill.

Big O de memoria

Medimos memoria auxiliar: estructuras nuevas que crea el algoritmo.

$O(1)$

Pocas variables cuyo número no crece con n .

contador, suma, candidato...

$O(n)$

Estructuras que pueden crecer con n .

set, dict, lista auxiliar, copia...

Entrada $\neq$ auxiliar

La lista recibida ya existía; contamos la memoria NUEVA que agregamos.

Trade-off: tiempo vs. memoria

Más memoria

set/dict suelen acelerar búsquedas.

Menos memoria

Matemáticas, invariantes o más comparaciones.

No hay regla universal

$O(1)$ espacio puede costar $O(n^2)$ tiempo.

Ejercicio · Missing Number

[3,0,1] → 2

Con memoria extra

```
# No válida A: buscar cada valor de 0 a n en la lista.  
# Tiempo  $O(n^2)$ : "valor in nums" recorre toda la lista,  
# y lo hacemos  $n + 1$  veces.  
def numero_faltante_dos_ciclos(nums):  
    for valor in range(len(nums) + 1):  
        if valor not in nums:  
            return valor  
  
# No válida B: guardar los números en un set.  
# Tiempo  $O(n)$ , pero memoria  $O(n)$  por el set.  
def numero_faltante_con_set(nums):  
    vistos = set()  
    for numero in nums:  
        vistos.add(numero)  
  
    for valor in range(len(nums) + 1):  
        if valor not in vistos:  
            return valor
```

set: $O(n)$ espacio

Con menos memoria

suma: $O(1)$ espacio

Ejercicio · Max Profit

[7,1,5,3,6,4] → 5

Con memoria extra

```
# No válida: probar todas las parejas (día de compra,  
# día de venta posterior).  
# Memoria  $O(1)$ , pero tiempo  $O(n^2)$  por los dos ciclos  
# anidados.  
def mejor_ganancia_dos_ciclos(precios):  
    mejor = 0  
  
    for compra in range(len(precios)):  
        for venta in range(compra + 1, len(precios)):  
            ganancia = precios[venta] - precios[compra]  
            if ganancia > mejor:  
                mejor = ganancia  
  
    return mejor
```

lista auxiliar: $O(n)$ espacio

Con menos memoria

dos variables: $O(1)$ espacio

Ejercicio · Majority Element

[2,2,1,1,1,2,2] → 2

Con memoria extra

```
# No válida A: contar cuántas veces aparece cada número
# con un diccionario.
# Tiempo O(n), pero memoria O(n) por el diccionario.
def elemento_mayoritario_con_diccionario(nums):
    conteo = {}

    for numero in nums:
        if numero in conteo:
            conteo[numero] = conteo[numero] + 1
        else:
            conteo[numero] = 1

    for numero in conteo:
        if conteo[numero] > len(nums) / 2:
            return numero
```

dict: O(n) espacio

Con menos memoria

Boyer-Moore: O(1) espacio

Ejercicio · Two Sum

[2,7,11,15], target=9 → True

Con memoria extra

```
# No válida A: probar todas las parejas.  
# Memoria  $O(1)$ , pero tiempo  $O(n^2)$ .  
def dos_sumas_dos_ciclos(nums, objetivo):  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] + nums[j] == objetivo:  
                return [i, j]
```

set: $O(n)$ espacio, $O(n)$ tiempo

Con menos memoria

sin set: $O(1)$ espacio, $O(n)$ tiempo

Ejercicio · Find the Duplicate Number

[1,2,3,1] → True

Con memoria extra

```
# No válida A: comparar cada par.
# No modifica nums y usa memoria O(1), pero tiempo O(n²).
def encontrar_repetido_dos_ciclos(nums):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                return nums[i]

# No válida B: guardar en un set los números que ya vimos.
# Tiempo O(n), pero memoria O(n) por el set.
def encontrar_repetido_con_set(nums):
    vistos = set()
    for numero in nums:
        if numero in vistos:
            return numero
        vistos.add(numero)
```

set: O(n) espacio, O(n) tiempo

Con menos memoria

sin set: O(1) espacio, O(n log n) tiempo

Qué aprendimos de los ejercicios

Matemáticas

Missing Number elimina una estructura auxiliar usando una fórmula.

Invariante

Majority y Max Profit conservan únicamente el estado necesario.

Sacrificar tiempo

Two Sum puede ahorrar memoria usando más comparaciones.

Contenido oficial: expresiones y operadores

Aritméticos

+
-
/
//
%
**

Relacionales

==
!=
>
<
>=
<=

Lógicos / asignación

and
or
not
=
+=
-=
*=
/=

Expresiones

Válida

$(a + b) * 2$

Produce un valor.

Inválida

$a + ($

Está incompleta: `SyntaxError`.

Ejemplo

$\text{base} * \text{altura} / 2$

Operadores aritméticos

$+$
 $-$
 $*$

suma · resta · multiplicación

$/$
 $//$
 $\%$

división · entera · residuo

$**$

potencia

Ejemplos

$7/2=3.5$ $7//2=3$ $7\%2=1$ $2**3=8$

Relacionales y lógicos

Comparaciones

$5 > 3 \rightarrow \text{True}$

$5 == 5 \rightarrow \text{True}$

$5 != 2 \rightarrow \text{True}$

Lógica

and: ambas

or: al menos una

not: invierte

Atención

= asigna · == compara

Asignación

Completa

```
total=10  
total=total+5  
total=total-2
```

Compacta

```
total=10  
total+=5  
total-=2
```

Resultado

Ambas terminan con total = 13.

Literales

Numéricas

5 · 3.14 · -2

Texto

'A' · "Diseño de algoritmos"

Booleanas

True · False

Precedencia y asociatividad

Precedencia

$$2 + 3 * 4 = 14$$

$$(2 + 3) * 4 = 20$$

Asociatividad

$$10 - 4 - 2 = 4$$

$$10 - (4 - 2) = 8$$

Consejo

Usa paréntesis para hacer explícita tu intención.

Práctica: predice antes de ejecutar

1-2

$5 + 2 * 3$
 $17 // 5$

3-4

$17 \% 5$
 $8 > 3$ and $2 == 2$

5-6

not ($10 < 4$)
 $x=5$; $x+=3$

Método

Predice → ejecuta → compara → explica.